Projet 3 : Diviser pour régner – Front de pareto

Le but de ce projet est de déterminer l'ensemble des points maximaux d'un ensemble de points du plan, correspondant au front de Pareto. Pour cela, nous avons mis en œuvre un algorithme de type « diviser pour régner » permettant de réduire la complexité du calcul par rapport à une approche naïve où on compare tous les points entre eux.

Définitions

Un point p de coordonnées (x_p, y_p) domine un point q de coordonnées (x_q, y_q) si et seulement si x_p > x_q et y_p > y_q. Un point est dit maximal s'il n'est dominé par aucun autre point de l'ensemble. L'ensemble des points maximaux constitue le front de Pareto. Dans ce projet, les points sont représentés par des couples (x, y) et sont triés selon l'abscisse x puis selon l'ordonnée y.

Justification théorique

Soit t le point de EPM(SR) ayant la plus petite abscisse. Comme l'ensemble des points est trié selon l'abscisse, tous les points de SR ont une abscisse strictement supérieure à celle des points de SL.

Si un point de EPM(SL) est dominé par un point de EPM(SR), alors ce point possède à la fois une abscisse et une ordonnée supérieures à celles de p. Le point t, qui est le point le plus à gauche parmi les points maximaux de SR, possède déjà une abscisse suffisante et, étant maximal, une ordonnée suffisamment grande.

Donc tout point de EPM(SL) dominé par un point de EPM(SR) est nécessairement dominé par le point t. Il suffit donc de comparer les points de EPM(SL) avec le point t pour éliminer ceux qui sont dominés.

Algorithme diviser pour régner

Définir fonction EPM(S):

```
    si Taille(S) <= 1 :
        retourner S
    m <- Taille(S) // 2
    SL <- S[1, … ,m]
    SR <- S[m+1, … ,n]

    EPM_L <- EPM(SL)
    EPM_R <- EPM(SR)

    t <- point de EPM_R avec la plus petite abscisse
```

```
pour chaque point p de EPM_L :
    si t domine p:
        supprimer p de EPM_L

retourner EPM_L + EPM_R
```

Analyse de la complexité

À chaque appel récursif, l'ensemble des points est divisé en deux sous-ensembles de taille quasiment égale. L'algorithme est ensuite appliqué récursivement sur chacun de ces sous-ensembles. Dans la phase de fusion, on parcourt les points maximaux du sous-ensemble gauche afin de supprimer ceux qui sont dominés par un point du sous-ensemble droit, cette tâche a une complexité de temps en O(n).

La relation de récurrence est donc :

$T(n) = 2T(n/2) + O(n)$

Cette relation correspond à un algorithme qui divise le problème en deux sous-problèmes de taille moitié et qui ensuite effectue un traitement proportionnel au nombre de points lors de la phase de fusion. Le nombre de niveaux de récursion est proportionnel à log n, et à chaque niveau le travail effectué est proportionnel à n. La complexité totale de l'algorithme est donc en O(n log n).

Implémentation et choix techniques

L'algorithme a été implémenté en Python en respectant le schéma diviser pour régner. Les points sont représentés par des couples d'entiers (x, y) stockés dans une liste. Les données sont triées selon l'abscisse avant l'exécution de l'algorithme, sinon l'algorithme ne peut pas fonctionner correctement. Le choix de la récursivité permet de traduire directement l'algorithme théorique en code, ce qui rend le programme plus lisible et plus conforme au raisonnement mathématique.

Test et résultats

L'algorithme a été testé sur les jeux de données fournis.

Pour le fichier data1 contenant 256 points, l'ensemble des points maximaux est constitué de 6 points comme prévu.

Pour le fichier data2 l'algorithme retourne 7 points maximaux.

Le fichier data3 contenant 100 000 points non triés a été correctement traité après avoir été trié, et le nombre de points maximaux obtenu est de 15.

Exécution du programme

Le programme s'exécute en lançant le fichier pareto.py. Les fichiers de données doivent se trouver dans le même dossier que le script (soit data1.txt, data2.txt, data3.txt).

Affichage du front de Pareto

Une représentation graphique des points et du front de Pareto a été réalisée à l'aide de la bibliothèque matplotlib. Les points maximaux apparaissent distinctement des autres points et forment une courbe correspondant au front de Pareto. L'affichage graphique a été réalisé pour les fichiers data1.txt et data2.txt et data3.txt.

Conclusion

Ce projet a permis de mettre en œuvre un algorithme de type diviser pour régner afin de calculer de façon efficace un front de Pareto. L'approche « diviser pour régner » permet de réduire énormément la complexité du calcul par rapport à une méthode naïve, tout en restant adaptée à de grands jeux de données. Les tests réalisés confirment la validité et l'efficacité de l'algorithme implémenté. La partie la plus complexe du projet a été la compréhension de la phase de fusion, en particulier le rôle du point t. Il permet de limiter considérablement le nombre de comparaisons.